

▼ PoData

```
import pandas as pd
data = pd.read_csv("/content/PoData.csv")
data.head()
```

	Unnamed: 0	accommod	account	action	activ	addit	adjust	agenc	anticip	appropri	...	unemploy	vice	vote	warrant	wide	will
0	1	3	1	2	3	0	0	3	3	4	...	1	1	2	1	1	6
1	2	0	1	1	3	1	2	1	0	1	...	0	1	0	0	1	8
2	3	0	0	2	2	1	1	2	3	0	...	0	1	1	2	0	8
3	4	3	1	1	2	1	1	1	0	3	...	0	1	1	0	1	7
4	5	2	0	2	1	1	0	3	3	0	...	2	1	2	2	0	4

▼ New Section

```
data = data.drop("Unnamed: 0", axis=1)
```

```
data.head()
```

	accommod	account	action	activ	addit	adjust	agenc	anticip	appropri	assess
0	3	1	2	3	0	0	3	3	4	3
1	0	1	1	3	1	2	1	0	1	2
2	0	0	2	2	1	1	2	3	0	0
3	3	1	1	2	1	1	1	0	3	3
4	2	0	2	1	1	0	3	3	0	0

5 rows × 11 columns

```
data['policy'].value_counts()
```

```
flat    106
up       20
lower    10
raise     9
down      5
Name: policy, dtype: int64
```

```
from collections import Counter
counter = Counter(data['policy'])
max_val = float(max(counter.values()))
class_weights = {class_id : max_val/num_images for class_id, num_images in counter.items()}
```

```
X = data.drop("policy", axis=1)
y = data['policy'].map({'flat':0, 'up':1, 'lower':2, 'raise':3, 'down':4 })
```

```
X.shape
```

```
(150, 140)
```

```
class_weights={
    0:1.0,
    1:5.3,
    2:10.6,
    3:11.77777777,
    4:21.2
}
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
import tensorflow as tf
from tensorflow.keras.layers import Dense
from keras.optimizers import Adam
from keras import Sequential
model = Sequential()
model.add(Dense(140, activation='relu', input_shape=(140,)))
model.add(Dense(128, activation="relu"))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='categorical_crossentropy',
optimizer='sgd',
metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=1, verbose=1, class_weight = class_weights)

Epoch 1/10
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
return dispatch_target(*args, **kwargs)
120/120 [=====] - 1s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 2/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 3/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 4/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 5/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 6/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 7/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 8/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 9/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
Epoch 10/10
120/120 [=====] - 0s 2ms/step - loss: nan - accuracy: 0.7083
<keras.src.callbacks.History at 0x7ceef4cb0a60>
```

```
model.evaluate(X_test, y_test)
```

```
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
return dispatch_target(*args, **kwargs)
1/1 [=====] - 0s 313ms/step - loss: nan - accuracy: 0.7000
[nan, 0.699999988079071]
```

▼ EcoData

```
data = pd.read_csv("/content/EcoData.csv")
data.head()
```

```
Unnamed: 0  accommod  account  action  activ  addit  adjust  agenc  anticip  appropri
```

	0	accommod	account	action	activ	addit	adjust	agenc	anticip	appropri
0	1	3	1	2	3	0	0	3	3	4
1	2	0	1	1	3	1	2	1	0	1
2	3	0	0	2	2	1	1	2	3	0
3	4	3	1	1	2	1	1	1	0	3
4	5	2	0	2	1	1	0	3	3	0

```
data.shape
```

```
(150, 142)
```

```
data = data.drop("Unnamed: 0", axis=1)
```

```
data['egrowth'].value_counts()
```

```
up      79
flat    36
down    35
Name: egrowth, dtype: int64
```

```

from collections import Counter
counter = Counter(data['egrowth'])
max_val = float(max(counter.values()))
class_weights = {class_id : max_val/num_images for class_id, num_images in counter.items()}

```

```

X = data.drop("egrowth", axis=1)
y = data['egrowth'].map({'flat':0, 'up':1, 'down':2 })

```

```
X.shape
```

```
(150, 140)
```

```
class_weights
```

```
{'up': 1.0, 'down': 2.257142857142857, 'flat': 2.1944444444444446}
```

```

class_weights = {
    1:1.0,
    2:2.2571428,
    0:2.194444
}

```

```

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

```

```

import tensorflow as tf
from tensorflow.keras.layers import Dense
from keras.optimizers import Adam
from keras import Sequential
model = Sequential()
model.add(Dense(140, activation='tanh', input_shape=(140,)))
model.add(Dense(128, activation="tanh"))
model.add(Dense(128, activation="tanh"))
model.add(Dense(128, activation='tanh'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='categorical_crossentropy',
optimizer='sgd',
metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=1, verbose=1, class_weight = class_weights)

```

```

Epoch 1/10
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
return dispatch_target(*args, **kwargs)
120/120 [=====] - 1s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4583
Epoch 2/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.4667
Epoch 3/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.3833
Epoch 4/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.2417
Epoch 5/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.2500
Epoch 6/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.4000
Epoch 7/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.4750
Epoch 8/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.5167
Epoch 9/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.5250
Epoch 10/10
120/120 [=====] - 0s 4ms/step - loss: 0.0000e+00 - accuracy: 0.4917
<keras.src.callbacks.History at 0x7ceee31ab910>

```

```
model.evaluate(X_test, y_test)
```

```

1/1 [=====] - 0s 169ms/step - loss: 0.0000e+00 - accuracy: 0.5333
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
return dispatch_target(*args, **kwargs)
[0.0, 0.5333333611488342]

```

▼ ElInflaData

```
data = pd.read_csv("/content/EInflaData.csv")
data.head()
```

	Unnamed: 0	accommod	account	action	activ	addit	adjust	agenc	anticip	appropri
0	1	3	1	2	3	0	0	3	3	4
1	2	0	1	1	3	1	2	1	0	1
2	3	0	0	2	2	1	1	2	3	(
3	4	3	1	1	2	1	1	1	0	3
4	5	2	0	2	1	1	0	3	3	(

```
data = data.drop("Unnamed: 0", axis=1)
```

```
data['inflation'].value_counts()
```

```
flat    65
down    49
up       36
Name: inflation, dtype: int64
```

```
X = data.drop("inflation", axis=1)
y = data['inflation'].map({'flat':0, 'up':1, 'down':2 })
```

```
from collections import Counter
counter = Counter(data['inflation'])
max_val = float(max(counter.values()))
class_weights = {class_id : max_val/num_images for class_id, num_images in counter.items()}
```

```
class_weights
```

```
{'down': 1.3265306122448979, 'flat': 1.0, 'up': 1.8055555555555556}
```

```
class_weights = {
    1:1.805555,
    2:1.3265,
    0:1.0
}
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
import tensorflow as tf
from tensorflow.keras.layers import Dense
from keras.optimizers import Adam
from keras import Sequential
model = Sequential()
model.add(Dense(140, activation='relu', input_shape=(140,)))
# model.add(Dense(140, activation="relu"))
# model.add(Dense(140, activation="relu"))
model.add(Dense(64, activation="tanh"))
model.add(Dense(64, activation='selu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='categorical_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=20, batch_size=1, verbose=1)
```

```
Epoch 1/20
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
return dispatch_target(*args, **kwargs)
120/120 [=====] - 1s 3ms/step - loss: 0.0000e+00 - accuracy: 0.3917
Epoch 2/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 3/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 4/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 5/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 6/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
```

```
Epoch 7/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 8/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 9/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 10/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 11/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 12/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 13/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 14/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 15/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 16/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 17/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 18/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 19/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
Epoch 20/20
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.4333
<keras.src.callbacks.History at 0x7ceed4b54970>

model.evaluate(X_test, y_test)

1/1 [=====] - 0s 156ms/step - loss: 0.0000e+00 - accuracy: 0.4333
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
    return dispatch_target(*args, **kwargs)
[0.0, 0.43333333373069763]
```

EmpData

```
data = pd.read_csv("/content/EmpData.csv")
data.head()
```

	Unnamed: 0	accommod	account	action	activ	addit	adjust	agenc	anticip	appropri
0	1	3	1	2	3	0	0	3	3	4
1	2	0	1	1	3	1	2	1	0	1
2	3	0	0	2	2	1	1	2	3	0
3	4	3	1	1	2	1	1	1	0	3
4	5	2	0	2	1	1	0	3	3	0

```
data = data.drop("Unnamed: 0", axis=1)

data['empGrowth'].value_counts()

up      76
flat    44
down    30
Name: empGrowth, dtype: int64

X = data.drop("empGrowth", axis=1)
y = data['empGrowth'].map({'flat':0, 'up':1, 'down':2 })

from collections import Counter
counter = Counter(data['empGrowth'])
max_val = float(max(counter.values()))
class_weights = {class_id : max_val/num_images for class_id, num_images in counter.items()}

class_weights
```

```
{'up': 1.0, 'down': 2.533333333333333, 'flat': 1.7272727272727273}

class_weights = {
    1:1.0,
    2:2.5333333,
    0:1.7272
}

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

import tensorflow as tf
from tensorflow.keras.layers import Dense
from keras.optimizers import Adam
from keras import Sequential
model = Sequential()
model.add(Dense(140, activation='tanh', input_shape=(140,)))
model.add(Dense(140, activation="tanh"))
model.add(Dense(140, activation="relu"))
model.add(Dense(64, activation="tanh"))
# model.add(Dense(64,activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='categorical_crossentropy',
optimizer='sgd',
metrics=['accuracy'])
model.fit(X_train, y_train,epochs=10, batch_size=1, verbose=1, class_weight=class_weights)

Epoch 1/10
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
return dispatch_target(*args, **kwargs)
120/120 [=====] - 1s 3ms/step - loss: 0.0000e+00 - accuracy: 0.3333
Epoch 2/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.3917
Epoch 3/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.3333
Epoch 4/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.3167
Epoch 5/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.3750
Epoch 6/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.3750
Epoch 7/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.3667
Epoch 8/10
120/120 [=====] - 0s 2ms/step - loss: 0.0000e+00 - accuracy: 0.5167
Epoch 9/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.5333
Epoch 10/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.4750
<keras.src.callbacks.History at 0x7ceecef73eb0>
```

```
model.evaluate(X_test, y_test)

1/1 [=====] - 0s 225ms/step - loss: 0.0000e+00 - accuracy: 0.5000
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
return dispatch_target(*args, **kwargs)
[0.0, 0.5]
```

▼ mRateData

```
data = pd.read_csv("/content/mRateData.csv")
data.head()
```

Unnamed: 0	accommod	account	action	activ	addit	adjust	agenc	anticip	appropri
0	1	3	1	2	3	0	0	3	3
1	2	0	1	1	3	1	2	1	0
2	3	0	0	2	2	1	1	2	3
3	4	3	1	1	2	1	1	1	0
4	5	2	0	2	1	1	0	3	3

```

data = data.drop("Unnamed: 0", axis=1)

data['tone'].value_counts()

dove      105
hawk       44
dove        1
Name: tone, dtype: int64

data['tone'].unique()

array(['dove', 'hawk', 'dove'], dtype=object)

for i in range(data.shape[0]):
    if data['tone'][i] == "dove ":
        data['tone'][i]="dove"
data['tone'].value_counts()

<ipython-input-146-6b62581853f6>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
data['tone'][i]="dove"
dove      106
hawk       44
Name: tone, dtype: int64

```

```

X = data.drop("tone", axis=1)
y = data['tone'].map({'dove':0,'hawk':1})

from collections import Counter
counter = Counter(data['tone'])
max_val = float(max(counter.values()))
class_weights = {class_id : max_val/num_images for class_id, num_images in counter.items()}
class_weights

{'dove': 1.0, 'hawk': 2.409090909090909}

class_weights = {
    0:1.0,
    1:2.409
}

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

import tensorflow as tf
from tensorflow.keras.layers import Dense
from keras.optimizers import Adam
from keras import Sequential
model = Sequential()
model.add(Dense(140, activation='tanh', input_shape=(140,)))
model.add(Dense(140, activation="tanh"))
model.add(Dense(140, activation="relu"))
model.add(Dense(64, activation="tanh"))
# model.add(Dense(64,activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='categorical_crossentropy',
optimizer='sgd',
metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=1, verbose=1, class_weight=class_weights)

Epoch 1/10
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e
return dispatch_target(*args, **kwargs)
120/120 [=====] - 2s 3ms/step - loss: 0.0000e+00 - accuracy: 0.7000
Epoch 2/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.7083
Epoch 3/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.7083
Epoch 4/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.7083
Epoch 5/10
120/120 [=====] - 0s 4ms/step - loss: 0.0000e+00 - accuracy: 0.7083
Epoch 6/10
120/120 [=====] - 0s 4ms/step - loss: 0.0000e+00 - accuracy: 0.7083
Epoch 7/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.7083

```

```
Epoch 8/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.7083
Epoch 9/10
120/120 [=====] - 0s 4ms/step - loss: 0.0000e+00 - accuracy: 0.7083
Epoch 10/10
120/120 [=====] - 0s 3ms/step - loss: 0.0000e+00 - accuracy: 0.7083
<keras.src.callbacks.History at 0x7ceeceee0190>
```

```
model.evaluate(X_test, y_test)
```

```
/usr/local/lib/python3.10/dist-packages/tensorflow/python/util/dispatch.py:1260: SyntaxWarning: In loss categorical_crossentropy, e>
return dispatch_target(*args, **kwargs)
WARNING:tensorflow:5 out of the last 5 calls to <function Model.make_test_function.<locals>.test_function at 0x7ceecf0212d0> trigger
1/1 [=====] - 0s 276ms/step - loss: 0.0000e+00 - accuracy: 0.7000
[0.0, 0.6999999888079071]
```

